Procedural Map Generation and AI Evaluation in a Turn-Based Gridworld

Luc van Driel, Yiwen Sun, Rwik Kamilya, Joris Lans, Raoul Schipper, and Binghao Guo

LIACS, Leiden University, Leiden, The Netherlands

Abstract. We present a simple gridworld game with a custom UI where the player or agent must eliminate all enemies within a number of turns, using different types of terrain and items. To assess the performance of both the agent and different map types of varying size, we implemented three types of agents: random, heuristic, and MCTS to measure their performance in different maps and at the same time measure the solvability of the different map types and sizes. The map sizes consist of small (9x7), medium (13x11), and large (17x13) and map types consist of baseline, random walk, and arena. During testing, we not only tuned these map settings, but also the player HP, enemy HP, enemy profile, item profile, and wall profile to aggregate the results and expose the spread in performance. We found that MCTS generally outperforms heuristic, which outperforms random. One of the two most important findings is that only medium and large maps are useful for evaluating performance between agents, since small maps are mostly trivial and all agents except for random achieve perfect win rates. On larger maps, heuristic win rates are around $35-37\%$, while MCTS-medium reaches around $79-83\%$, giving a gap of about $41-47$ percentage points. The other one is that win rate does not tell the full story. The type of map has an effect on the length of any game, where random walks with more chokepoints cause longer games. This information is useful for further balancing improvements of the game.

Keywords: Gridworld · MCTS · Game balancing

1 Introduction

Procedural content generation is commonly used in games to create levels, maps, and scenarios with less manual design effort [4,5]. However, generated content still needs to be evaluated, because a generated map is not necessarily playable, solvable, or interesting. This project studies this problem in a small turn-based gridworld, where procedural map generation is combined with automated AI-based evaluation.

The project focuses on a simple tactical game environment. The environment is designed as a controllable testbed with a player unit, enemies, obstacles, terrain features, and a clear win condition. This limited scope makes it possible to

generate maps systematically and evaluate them under repeated experimental conditions.

The main goal of the project is to investigate how generated maps influence gameplay, difficulty, and solvability. To do this, the project consists of three main components: a gridworld environment, a procedural map generation system, and AI agents for automated evaluation. The generated maps are evaluated by agents such as a heuristic baseline and a Monte Carlo Tree Search agent.

This setup allows the AI agents to serve two roles. First, they act as players that attempt to solve the generated maps. Second, their performance is used as an evaluation signal for the generated content. Measures such as win rate, number of turns, damage taken, and variation in outcomes can be used to compare different maps or generation settings.

The project is connected to several topics from the Modern Game AI Algorithms course. It uses procedural content generation to create maps, search-based game AI through Monte Carlo Tree Search, and empirical evaluation to compare agent performance across generated content [7]. The report describes the game environment, the map-generation approach, the AI agents, the experimental setup, and the strengths and limitations of the resulting system.

This project connects naturally to several course topics. Procedural content generation is reflected in the automatic generation of playable maps with different structural properties. Search-based game AI is represented by the use of Monte Carlo Tree Search (MCTS) and heuristic agents for automated evaluation. The project also relates to learning and optimization through the design and comparison of increasingly effective agents. In addition, it connects to computer experimentation through repeated runs and metric-based comparison of map difficulty and agent performance.

2 Game Design and Environment

The game is a small-scale turn-based tactical shooter in a 2D gridworld, implemented using Griddly as the underlying gridworld environment platform [1]. The player can move and shoot each turn and use items and the terrain to increase its chances of winning by defeating all enemies.

2.1 Game Rules

Each round has a fixed sequence whereby every action is optional:

1. The player activates an item (only possible if the player is in possession of an item)
2. The player moves
3. The player attacks
4. All the enemies take their turn using breadth-first search (BFS), only making valid moves; they cannot walk through walls or other enemies.

Table 1. Available map size presets. Dimensions are given as width × height in tiles, including the surrounding wall border.

Preset	Dimensions	Playable area
small	9×7	7×5
medium	13×11	11×9
large	17×13	15×11

The gridworld can be of different sizes depending on the preset. The options are specified in Table 1. Movement is allowed in all vertical and horizontal directions, so up, down, left, or right, and is blocked only by walls and enemies.

The player's health is set at 4 HP by default, whereas the enemies all have 2 HP. Both the player and the enemies have a standard attack range and damage of 1. Attacks are, like movement, only legal in the vertical and horizontal directions where walls always block an attack. Whenever the HP of any unit reaches 0, the unit dies, causing the enemy to die or the game to stop when this happens to the enemy or player, respectively.

2.2 Terrain and Items

The game contains several terrain bonuses that are activated when the player stands on the same spot where the bonus is located:

- **Bunker.** Ending a turn in a bunker will absorb exactly one damage for a single turn and then disappear.
- **Bush.** When a player ends its turn in a bush location, it receives a 50% chance of fully negating the enemy's attack.
- **Hill.** On a hill, the attack range increases by 1. This allows players to hit enemies that are further away, still not allowing for diagonal attacks.

The game also contains items that can be picked up by walking onto a tile that contains any item. After picking up the item, it can be activated once, and the effect persists for a number of turns that differs per item.

- **Dual berettas.** Activating the Dual berettas allows a player to shoot twice within the same turn. This could be to either the same or two different enemies. The item has a duration of five turns.
- **Golden gun.** A shot with this gun guarantees a one-hit kill. However, it can only be used once before it disappears from the player's inventory.
- **Shotgun.** Shooting with the shotgun equipped allows the player to shoot at enemies at range 2 instead of 1. When an enemy is right next to the player, the shotgun will do double damage. The item has a duration of five turns.
- **Vehicle.** Activating the vehicle allows the player to move two cells in the same turn. The item has a duration of three turns.

2.3 Win and Loss Conditions

The game can be either won or lost. The game is won if the player eliminates all enemies before the turn cap is reached. Losing can be done in two ways; the player's HP reaches 0 or the turn cap is reached. This turn cap is set at 40 for the demo and at 120 for the experiments.

2.4 Enemy design

Enemies have the same move set as the player, but in each turn, they have the option to either move or attack. When located next to the player, an enemy will always try to damage the player. Enemies cannot pick up items or benefit from terrain bonuses.

2.5 Pygame User Interface

To make the prototype easier to play and debug, we implemented a Pygame user interface on top of the original command-line version. This made it easier to inspect the game state during development and to present the system in a more readable, game-like form.

The interface shows the board, the player, enemies, terrain features, and items in a single view. It also displays useful game-state information such as the current turn, remaining health, collected items, active effects, and the currently planned action. Interaction is mouse-based: the player can select the unit, preview movement, inspect possible attacks, and confirm a turn directly through the interface. This makes terrain effects and item usage easier to follow during play.

The Pygame interface is mainly intended as a human-playable front end for the same environment that is used in the experiments. While it is not part of the PCG or AI logic itself, it helps make the rules, tactical choices, and current game state easier to understand.

3 Procedural Map Generation

This project uses procedural content generation (PCG) to create playable tactical grid maps automatically. Instead of relying only on hand-designed levels, the system can generate maps with different sizes, layouts, enemy counts, wall densities, and item densities. The purpose of this component is to support repeatable AI evaluation under varied map conditions. Each generated level is checked for structural validity before it is used, so the agent is not evaluated on impossible or trivial maps.

Fig. 1. Pygame-based user interface of the GridBattle prototype. The interface shows the board, units, terrain features, items, and game-state information in a single tactical view.

3.1 Map Representation

The game uses two levels of map representation. Internally, each level is stored as a two-dimensional tile grid encoded by a multiline character string. Symbols such as `W`, `.`, `A`, and `E` represent walls, empty floor tiles, the player, and enemies respectively. Other symbols represent terrain and items.

This character-based format is mainly used as an intermediate representation for procedural generation, validation, and conversion into the game environment. It is not the final player-facing interface. As described in the game environment section, the playable version includes a Pygame-based user interface that renders the board, units, terrain features, items, and game-state information in a visual tactical view. This separation allows the generator to operate on a compact symbolic grid, while the UI presents the same underlying map in a more readable and playable form.

All generated maps are rectangular and surrounded by boundary walls. Before a map is loaded, the layout parser checks that the level is non-empty and that all rows have the same width. This avoids malformed maps and ensures that both the environment logic and UI renderer receive a consistent grid structure.

3.2 Generation Method

The project supports three procedural map generators: `baseline`, `random_walk`, and `arena`. All three follow the same general pipeline: a candidate layout is generated, actors and objects are placed, and the result is checked for validity before being accepted.

The `baseline` generator creates a standard random obstacle map with boundary walls, randomly placed player and enemies, and interior obstacles controlled by obstacle density. The `random_walk` generator starts from a wall-filled grid and carves floor tiles through a constrained random walk, producing corridors, branches, chambers, dead ends, and chokepoints. The `arena` generator creates a more open combat map with sparse mirrored obstacles and a player position near the center. Therefore, the generator is best understood as a parameterized generate-and-test PCG system rather than a search-based PCG optimizer [5,4].

3.3 Validity Checks

Generated maps are not applied automatically. Each candidate must pass a structural validity check to avoid impossible or uninteresting maps. The code uses BFS to compute which tiles are reachable from the player's starting position while treating walls and enemies as blocked positions.

For each enemy, the generator checks whether there is at least one reachable tile from which the player can attack that enemy. A map is rejected if any enemy is unreachable or cannot be attacked. The reachable region must also be large enough: it must contain at least four tiles and at least one third of all non-wall tiles. These checks ensure basic playability, although they do not guarantee balanced difficulty.

3.4 Generation Parameters

The generator can be controlled through several experimental parameters. The map size can be `small`, `medium`, or `large`, with specific sizes, specified in Table 1. Map type can be `baseline`, `random_walk`, or `arena`. Enemy count can be adjusted through levels such as `low`, `default`, `medium_plus`, and `high`.

Wall density can be set to `low`, `default`, or `high`. For baseline and arena maps, this directly changes obstacle density; for random-walk maps, it changes the floor fraction. Item level can be `few`, `normal`, `many`, or `none`, controlling how many terrain features and items are placed. Finally, a random seed is used to make generated maps reproducible for experiments.

4 AI Agents

We use three agents to play the game. There is a random agent, a heuristic agent, and a Monte Carlo Tree Search (MCTS) agent. The MCTS agent is run with different search budgets, so we end up with a small, a medium, and a strong version.

All three agents look the same from the outside. Each turn they receive a `BattleSnapshot`, which is simply a particular game state. It shows the grid, the player, the enemies, the terrain (hills, bushes, bunkers), the items on the map, the player's inventory, and any active item effects. The agent then returns a `TurnAction` that packs a whole turn into one object: an optional item activation, a move (one tile, or two when the vehicle is equipped), an optional primary attack, and an optional secondary attack used only by the dual berettas. These fields are resolved in the same order the game itself uses (activate, move, attack) so the agent and the game can never disagree about the phase order.

4.1 Random Agent

The random agent is only there as a lower bound baseline. Each turn, it picks a move at random from the legal moves at its current location. Then, it picks an attack at random from the attacks available after that move, including "do not attack". It also activates one of its unused items with a 50% chance. Because it never plans, its win rate measures how often a map can be cleared by chance alone. This gives a floor that the other agents should comfortably beat, and it flags maps that are trivially winnable regardless of skill.

4.2 Heuristic Baseline Agent

The heuristic agent follows a simple "approach and shoot" idea: each turn it fires if an enemy is already in range, and otherwise steps one tile closer to the nearest position from which it could fire. It does not search. The agent is deterministic, which means that given the same snapshot, it is guaranteed to pick the same action. This makes it a stable baseline and a sensible rollout policy for the MCTS: the search plays out its simulation phase with this heuristic instead of with random moves. The agent does three things every turn:

1. **Activate an item.** If it has an unused item, it activates one, specifically the first unused item in its inventory. Items with a duration (`dual_berettas`, `shotgun`, `vehicle`) are preferred over one-shot items.
2. **Attack if you can.** For every enemy, it checks if it can attack from where it is now. If at least one enemy is in range and there is a clear line of sight, it attacks and does not move that turn.
3. **Otherwise, walk closer.** If no enemy is in range, it works out all the tiles from which it could shoot an enemy. Then it uses BFS to find the closest one and walks one step along that path. After moving, it tries to attack again. If BFS finds no path (for example, the player is fully blocked in), it just takes the first cardinal step that is not blocked.

When the `dual_berettas` item is active, the agent also picks a second attack against a different enemy if one is in range. The heuristic is simple on purpose. It has no knowledge of damage trades, chokepoints, or how good a piece of terrain is. It also never moves away from an enemy.

4.3 Monte Carlo Tree Search Agent

The MCTS agent uses single-player UCT (Upper Confidence bounds applied to Trees) on the gridworld [3,2]. It lives in `mcts.py` and uses a separate forward model in `simulator.py` for planning its moves, so the real game state is untouched during search.

Action space. At each node, the legal `TurnActions` are every combination of three choices:

- *Item activation*: "do not activate" plus one option for each unused item in the inventory.
- *Move sequence*: "stay" plus every legal one step move from the spot after activations. If the vehicle is active, we also add every legal two step move from that spot.
- *Attack*: "do not attack" plus one `PhaseAction` per enemy that can be reached from where the player ends up after the move. When `dual_berettas` is active, the agent also adds a secondary attack against a different enemy.

Item activation is part of the planned turn, not something that happens in the background. That way, the search can learn when to spend a one-shot item like the golden gun and when to save it for later.

Forward model. The forward model in `simulator.py` plays out one full round in the same order as the real game. It is a planning model and not a full replacement for the game, but it is high fidelity on purpose, modelling items and terrain explicitly rather than abstracting them away:

1. The chosen item, if any, is moved from the inventory to the active effects list with the appropriate duration.
2. The player applies the move sequence. The vehicle effect allows two tiles in one turn. A move into a wall or into another unit is silently skipped, just like in the real game.
3. If the tile the player ends up on has a map item, that item is picked up into the inventory.
4. The player attack sequence is resolved, applying each active item's effect (golden gun, shotgun, dual berettas) exactly as the real game does. Attacks check line of sight and cardinal direction with `has_clear_line` and `cardinal_relation`, so the simulator and the real game agree on which targets are actually hit.
5. Each enemy then runs BFS toward the player and either takes one step closer or attacks if it is already next to the player. Incoming damage is then modified by terrain: a bush gives a 50% chance to fully cancel the damage (this is the only random event in the model), and a bunker absorbs exactly one damage and then collapses.
6. Finally, every active effect has its duration reduced by one. Any effect that drops to zero is removed.

The forward model takes a random number generator that the MCTS agent reuses across its iterations. So, inside one search tree, the bush outcomes are consistent, but across separate trees, they can be different. Hill range bonuses do not need their own code path. The `get_unit_profile` call inside the simulator already adds the hill bonus when the player ends a turn on a hill.

Search procedure. The agent runs the normal four phase MCTS loop (selection, expansion, simulation, backpropagation) for a fixed number of iterations per turn. Selection follows the UCT rule

$$\mathrm{UCT}(s,a) = \bar{Q}(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}},$$

with exploration constant $c = \sqrt{2}$. Children that have never been tried get picked first. Expansion picks one untried action, applies the forward model, and adds the resulting state as a new child. The rollout policy is the heuristic agent above. Rollouts are cut off at a fixed depth that depends on the profile. If a rollout reaches a real game over, the value is $+1$ when the player killed all enemies and -1 when the player died. Otherwise the rollout is scored with this shaped heuristic

$$V(s) = -0.1\,|E(s)| - 0.02 \sum_{e \in E(s)} h_e + 0.05\,h_p,$$

where $E(s)$ is the set of remaining enemies, h_e their hit points, and h_p the players hit points. The shaping is small on purpose, so that real wins and losses are still what mostly drives the score. After all iterations are done, the root action with the most visits is picked. Ties are broken by the mean value. If the root expands no children at all (a guard that should not trigger in practice, since there is always at least the wait action to expand), the agent falls back to the heuristic for that turn.

Search profiles. We use the same `MctsAgent` class with different search budgets to compare a weak and a strong search. Table 2 lists them. These are the profiles used in Section 5 (`mcts_small`, `mcts_medium`). The `mcts_strong` profile exists in the code but is not used in the validation runs.

Table 2. MCTS search profiles. Exploration constant is $\sqrt{2}$ for all profiles. More iterations and a deeper rollout mean the agent thinks more per turn, but it also takes more real (wall-clock) time to compute each turn.

Profile	Iterations / turn	Rollout depth
`mcts_small`	50	10
`mcts_medium`	200	30
`mcts_strong`	800	40

You can pass in a random seed for a reproducible run. Once the seed is fixed, the order in which actions are tried at each node and the bush dodge outcomes inside the rollouts are both deterministic.

5 Experiments

This section evaluates whether the procedural generator produces configurations that are playable, meaningfully different in difficulty, and able to separate agents of different strength. The experiments vary three main PCG dimensions: map size, map type, and generator profiles. The size presets are `small`, `medium`, and `large`; the map types are `baseline`, `random_walk`, and `arena`. The remaining settings are expressed as profiles rather than raw generator parameters. Table 3 summarizes these profiles.

Table 3. Generator profiles used in the experiments.

Profile type	Values	Meaning
Enemy profile	`light`, `normal`, `medium_plus`, `heavy`	Controls enemy count relative to the map-type and size preset. `light` subtracts one enemy from the default, `normal` keeps the default, `medium_plus` adds one enemy (only on medium maps), and `heavy` adds two enemies. Counts are clipped to at least one enemy.
Item profile	`few`, `normal`, `many`	Controls the amount of terrain and items placed on free cells. The profiles correspond to approximately 6%/2.5%, 10%/4%, and 16%/6% terrain/item fractions, respectively.
Wall profile	`open`, `normal`, `tight`	Controls obstacle density through a wall-density multiplier. `open` lowers density, `normal` uses the generator default, and `tight` mainly increases obstacle density for the baseline generator while preserving the identity of the random-walk and arena generators.

The primary performance measure is win rate. We also recorded turn count, damage taken, final HP, remaining enemies, and structural map measures such as reachable floor fraction, interior wall density, dead ends, and chokepoints. These secondary measures are important because two configurations can have similar win rates while still producing different pacing or spatial structure.

5.1 Broad PCG Screening

The first experiment was a broad PCG screening run over the main generator factors. It crossed size, map type, enemy profile, item profile, wall profile, and

Table 4. Scale and purpose of the main experiment suites.

Experiment	Configs	Maps/config	Total maps	Purpose
PCG screening	648	20	12,960	Broad search over generator profiles
Medium enemy tuning	144	30	4,320	Tune the medium-map enemy profile
OFAT sensitivity	68	30	2,040	Local sensitivity around a tuned baseline
Final validation	36	50	1,800	Validate the selected final defaults

agent. The purpose of this run was exploratory: rather than selecting final settings directly, it was used to identify which parts of the generator had the largest effect on difficulty and where more targeted tuning was needed.

The screening results showed that difficulty was not controlled by a single variable. Map size had a clear global effect, but enemy profile, item availability, and wall structure interacted with both size and map type. Increasing enemy pressure generally reduced win rates, while additional items could compensate by giving stronger agents more tactical options. Wall structure affected pacing and encounter shape: constrained layouts tended to create longer games and more chokepoints rather than simply shifting all win rates up or down.

The most important outcome of the screening run was a gap in the medium-map difficulty range. Aggregating over map type, item profile, and wall profile, medium maps with `normal` enemies were close to saturated: the heuristic won 94.2% of episodes, MCTS-small won 99.2%, and MCTS-medium won 98.3%. The average gap between the heuristic and MCTS-medium was therefore only 4.2 percentage points, and all 18 `normal`-enemy medium configurations gave MCTS-medium at least a 95% win rate. In contrast, `heavy` enemies often made medium maps too punishing: the heuristic won only 5.6% of episodes, and MCTS-medium dropped to 41.9%. Moreover, 14 of the 18 `heavy`-enemy medium configurations had MCTS-medium win rates of 50% or lower. This suggested that `normal` enemies were too easy, while `heavy` enemies overshot the target difficulty, motivating a second, targeted tuning experiment with the intermediate `medium_plus` enemy profile.

5.2 Medium-Map Enemy Tuning

The medium-map tuning experiment focused on `medium` maps and compared three enemy profiles: `normal`, `medium_plus`, and `heavy`. For each enemy profile, we tested all three map types, two item profiles, and two wall profiles. The aim was to find settings where the heuristic agent was challenged but MCTS-medium could still succeed reliably.

Figure 2 shows the open-wall subset of this experiment. The large number in each cell is an auxiliary balance score used only for ranking candidate settings

Fig. 2. Medium-map enemy tuning for open-wall settings. Each cell shows an auxiliary balance score, with the heuristic and MCTS-medium win rates shown below as "H / M".

during tuning:

$$s = 1.0 - |w_M - 0.80| - 0.7|w_H - 0.50| + 0.6(w_M - w_H) - 0.5w_R,$$

where w_R, w_H, and w_M are the win rates of the random, heuristic, and MCTS-medium agents. The score favours settings where MCTS-medium is strong but not saturated, the heuristic is challenged but not completely failing, the MCTS–heuristic gap is large, and the random agent is unsuccessful. This score is not used as a final performance measure; it is a practical tuning criterion for selecting promising medium-map configurations.

The raw win rates in Figure 2 are more important than the exact score value. With `normal` enemies, several medium settings remain too easy. With `heavy` enemies, the heuristic is often pushed too low. The intermediate `medium_plus` profile creates a more useful middle region, especially when combined with `many` items. This led to the final medium-map setting:

$$\texttt{medium} \rightarrow \texttt{medium\_plus enemies}, \quad \texttt{many items}, \quad \texttt{open walls}.$$

5.3 One-Factor-at-a-Time Sensitivity

After choosing a promising medium-map baseline, we ran a one-factor-at-a-time sensitivity experiment. The baseline configuration was a medium baseline map with `medium_plus` enemies, `many` items, `open` walls, player HP 4, and enemy HP 2. Each sensitivity test changed one factor while keeping the rest fixed. The values in Table 5 are win-rate ranges, computed as the maximum minus the minimum win rate observed when varying that factor, including the baseline setting.

The strongest effects come from combat-rule parameters and enemy pressure. Enemy HP has the largest range for every non-random agent, and player HP also

Table 5. OFAT win-rate ranges by factor and agent, in percentage points. Darker cells indicate larger effects.

Factor	Random	Heuristic	MCTS-small	MCTS-medium
Enemy HP	53.3%	96.7%	83.3%	70.0%
Enemy profile	66.7%	86.7%	50.0%	33.3%
Player HP	10.0%	83.3%	50.0%	43.3%
Map size	66.7%	53.3%	33.3%	26.7%
Item profile	3.3%	20.0%	23.3%	30.0%
Wall profile	3.3%	16.7%	3.3%	20.0%
Map type	0.0%	6.7%	13.3%	20.0%

strongly affects the heuristic and MCTS agents. Among the generator-facing controls, enemy profile is the strongest local difficulty knob. Item profile, wall profile, and map type have smaller direct win-rate ranges, but they still influence the style of the generated encounters.

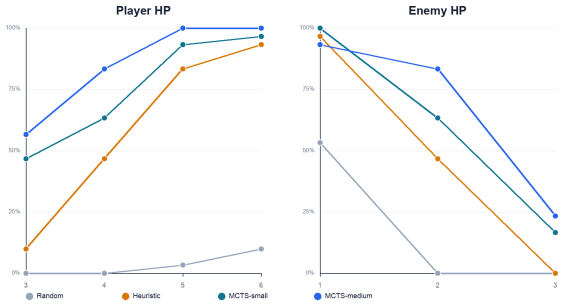

Fig. 3. Sensitivity of win rate to player HP and enemy HP. HP values strongly affect success rates, so the final validation keeps combat HP fixed.

Figure 3 shows the direction of the HP effects. Increasing player HP quickly increases win rate and can saturate the stronger agents. Increasing enemy HP has the opposite effect and sharply suppresses win rates. For this reason, the final validation keeps combat HP fixed and uses generator profiles rather than HP changes as the main balancing mechanism.

5.4 Final Validation

The final validation experiment evaluates only the selected generator defaults. Unlike the previous experiments, this run was not a search over settings. Its purpose was to measure the behaviour of the final configuration set with more episodes per cell.

The selected defaults were based on the previous experiments. Small maps use `normal` enemies, `normal` items, and `normal` walls for all map types. Medium

maps use the tuned setting of `medium_plus` enemies, `many` items, and `open` walls. Large maps use `normal` enemies and `many` items; the baseline generator keeps `normal` walls, while random-walk and arena maps use `open` walls.

Table 6. Aggregate final-validation win rates by map size. The gap is MCTS-medium minus heuristic.

Size	Random	Heuristic	MCTS-small	MCTS-medium	Gap
Small	62.7%	100.0%	100.0%	100.0%	0.0 pp
Medium	1.3%	37.3%	72.7%	78.7%	41.4 pp
Large	1.3%	35.3%	74.7%	82.7%	47.4 pp

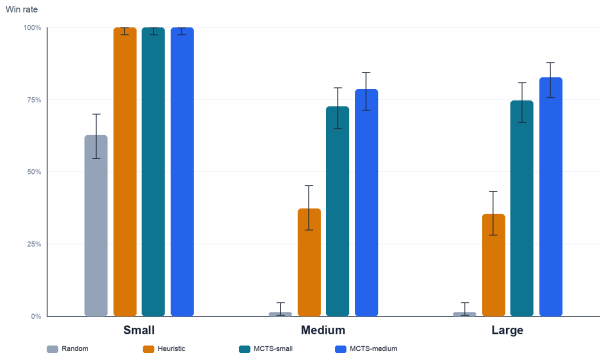

Fig. 4. Aggregate final-validation win rates by size and agent, with Wilson confidence intervals [6]. Small maps are easy, while medium and large maps separate heuristic and MCTS agents.

Table 6 and Figure 4 show that small maps form a clearly easy tier. The heuristic and both MCTS agents solve all small-map episodes, while the random agent wins 62.7%. Medium and large maps are not clearly separated by win rate. The heuristic wins 37.3% on medium maps and 35.3% on large maps, while MCTS-medium wins 78.7% and 82.7%, respectively. Thus, the final settings create one easy tier and two challenging tiers with similar win-rate difficulty.

However, medium and large maps differ in pacing. Table 7 shows that terminal turn counts increase with size for both the heuristic and MCTS-medium agents. The heuristic averages 5.6 turns on small maps, 12.5 on medium maps, and 16.3 on large maps. MCTS-medium takes significantly more turns and averages 11.1, 24.2, and 28.5 turns. Larger maps therefore do not necessarily reduce win rate further, but they tend to create longer problems.

Table 7. Secondary final-validation outcomes for the heuristic and MCTS-medium agents. Final HP and remaining enemies are averaged over all episodes, so losses contribute zero final HP.

Size	Agent	Win rate	Turns	Damage	Final HP	Enemies left
Small	Heuristic	100.0%	5.6	1.47	2.53	0.00
Small	MCTS-medium	100.0%	11.1	1.23	2.77	0.00
Medium	Heuristic	37.3%	12.5	3.52	0.48	0.87
Medium	MCTS-medium	78.7%	24.2	2.48	1.52	0.34
Large	Heuristic	35.3%	16.3	3.51	0.49	0.79
Large	MCTS-medium	82.7%	28.5	2.55	1.45	0.33

The combat-related metrics support the win-rate results. On medium and large maps, MCTS-medium takes less damage on average, finishes with more HP, and leaves fewer enemies alive when it loses. The turn counts are higher for MCTS-medium, but this is expected because MCTS-medium survives by taking a more tactical approach and wins many more episodes; the heuristic often terminates earlier by losing.

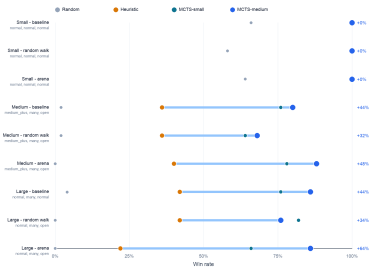

Fig. 5. Final-validation skill gaps by selected generator setting. The horizontal gaps show how much more often stronger agents solve the same type of generated content.

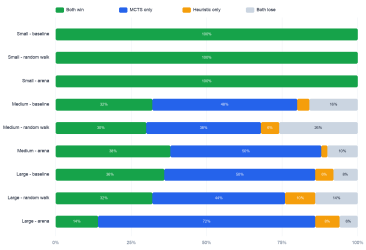

Fig. 6. Paired outcome comparison between heuristic and MCTS-medium on matched map seeds. The blue segment indicates maps won only by MCTS-medium.

Figure 5 gives the configuration-level view. The small configurations are saturated, so there is no visible gap between the heuristic and MCTS-medium. In contrast, medium and large settings show consistent separation. The largest gap occurs on large arena maps, where the heuristic has a low win rate but MCTS-medium remains highly successful. Interestingly, the MCTS-small agent outperforms MCTS-medium on large random-walk maps.

The paired outcome plot in Figure 6 controls for map randomness by comparing heuristic and MCTS-medium on matched seeds. Small maps fall entirely into the "both win" category. On medium and large maps, many seeds are won

only by MCTS-medium, while the reverse case is small. This confirms that the MCTS advantage is not merely caused by sampling easier maps; it solves more of the same generated content.

5.5 Structure and Pacing

The final validation also records structural map properties. To avoid counting the same generated map multiple times, the structural averages in Table 8 are computed after deduplicating by configuration and map seed. The turn column is the average terminal turn count over all agents for the same size and map type.

Table 8. Structural final-validation metrics by size and map type. Reachable floor and wall density are fractions; dead ends, chokepoints, and turns are averages.

Size	Map type	Reach. floor	Wall dens.	Dead ends	Chokepoints	Turns
Small	Baseline	0.960	0.092	1.24	1.78	9.5
Small	Random walk	0.938	0.539	2.54	10.18	11.6
Small	Arena	0.967	0.122	2.58	3.00	10.0
Medium	Baseline	0.964	0.097	2.30	3.52	14.2
Medium	Random walk	0.948	0.323	4.48	16.76	26.3
Medium	Arena	0.967	0.075	1.06	1.12	15.9
Large	Baseline	0.961	0.197	7.10	11.98	19.0
Large	Random walk	0.970	0.309	4.64	13.70	30.2
Large	Arena	0.974	0.076	2.24	2.86	20.5

Table 8 and Figure 7 show that random-walk maps are the clearest source of long, constrained games. They have the highest chokepoint counts in the small and medium settings and the longest average turn counts in all three sizes. This supports the interpretation that random-walk maps change pacing by creating corridor-like structure and bottlenecks.

The relationship between chokepoints and turns is not uniform across all map types. Arena maps do not follow the same pattern as clearly: they have low chokepoint counts, but medium and large arena maps can still take a substantial number of turns because of map size, open movement, and enemy pressure. Thus, chokepoints explain part of the pacing difference, especially for random-walk layouts, but they are not the only determinant of game length.

6 Discussion

The project succeeds at more than simply generating valid grid maps. The generator, agents, and evaluation pipeline together are analyzed and evaluated. Maps are procedurally produced, validated for structural playability, played repeatedly by agents of different strengths and compared with win rate, pacing, remaining

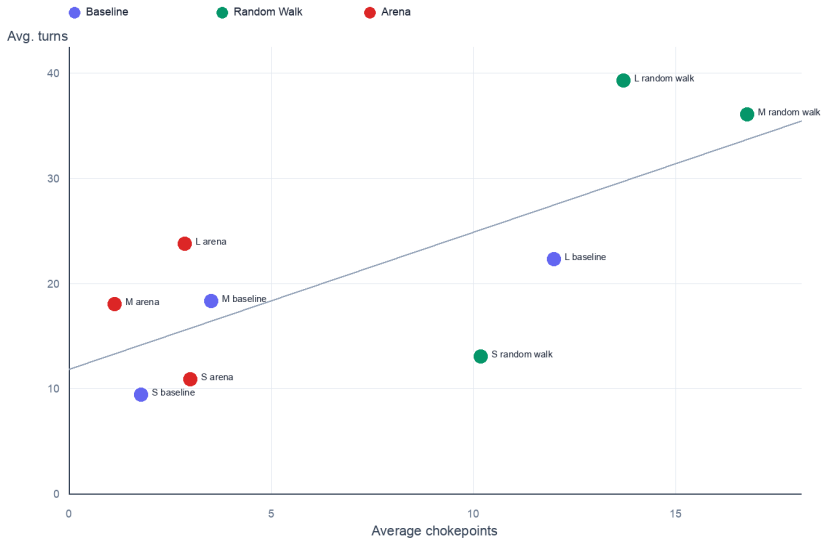

Fig. 7. Relationship between average chokepoints and average terminal turn count. Random-walk maps tend to create more chokepoints and longer games.

enemies, and topological measures. The generated content is treated as an object that can be measured or tuned.

Playability and difficulty are inherently different properties. Validity checks ensure that enemies can be reached and attacked, but this does not guarantee a balanced map for the agent. The broad screening experiment shows some configurations solved by almost all non-random agents while others were too difficult, even for MCTS-medium. Thus, with the later tuning experiments, we created the `medium_plus` enemy profile to bridge the gap between the `normal` and `heavy` enemy profiles.

As shown in Table 6 and Figure 4, small maps are saturated for the heuristic and MCTS agents and are therefore best interpreted as tutorial-like content. Medium and large maps are harder, separating local heuristic play from look-ahead-based planning. The paired outcome analysis in Figure 6 compares agents on matched map seeds. MCTS-medium solves many of the generated maps failed by the heuristic agent.

Medium and large maps are not cleanly ordered by win rate, but rather their difference is more visible in pacing. Table 8 and Figure 7 show that random-walk maps generate more choke-points with longer games, while arena maps are more open and direct. Thus, procedural parameters affect whether an agent wins and the kind of problems the agent has to solve. Desirable content is not just defined

by success probability alone, as a map can be fair but dull, hard but short or strategically rich without being much harder in aggregate.

We also compare agents across environment design. The heuristic agent is simple and locally sensible: it activates useful items, attacks when possible and otherwise moves toward attack positions. Its failures indicate where local action selection is not good enough. MCTS works better on medium and large maps as it can evaluate delayed consequences of positioning, movement or action. With MCTS-small outperforming MCTS-medium on large random-walk maps, search behavior is sensitive to budget, roll-out policy, stochasticity and the shape of the forward model.

6.1 Strengths

A major strength of this project is its full experimental pipeline. It does a broad screening, targeted tuning, one-factor-at-a-time sensitivity analysis and final validation. These ablations helped in developing the final configuration choices, making it more robust.

Another strength is the use of multiple agents to study behavior. The random agent is used as a low baseline, the heuristic agent displays cheap, local decision-making and the MCTS agents represent increasing amounts of search. Thus, we can study whether the maps are technically solvable and whether they reward deliberate play.

Using multiple-seed comparisons between heuristic and MCTS-medium agents on the same generated maps improves experimental rigor. It shows how MCTS solves content that the heuristic cannot.

Evaluation does not rely only on win rate. Secondary combat metrics and structural map metrics make the analysis more game-design oriented. This helps connect procedural generation to actual game-play qualities such as pacing, bottlenecks, and encounter length.

6.2 Weaknesses and Limitations

An apparent limitation is how intentionally small the game is. The gridworld abstracts away many sources of complexity that could be found in larger games, like, partial observability, long-term resource management, richer enemy behavior and more diverse objectives.

Another limitation is that MCTS uses a custom forward model rather than stepping the Griddly environment directly during search. Although the simulator models project-level mechanics used in the game, including terrain, items, active effects, enemy movement, among others, it is still a separate planning model. Small differences in action ordering, stochastic outcomes, enemy resolution, or terminal handling can affect the values estimated by MCTS. This is a common trade-off in game AI, where a compact forward model makes repeated look-ahead practical, but model fidelity becomes a part of the algorithmic error.

The experiments are limited by the chosen agents. the heuristic agent is a reasonable baseline, but it is only one hand-designed policy. The MCTS agents

use fixed budgets and a heuristic roll-out policy. Stronger planning variants, learned policies, or adversarial enemy policies could change which maps appear balanced.

The tuning score is another limitation. Although useful for ranking candidate settings, it encodes subjective design preferences, like, high but non-saturated MCTS performance, challenged heuristic performance, a visible skill gap and low random performance. Different design goals would require a different score. Thus the score should be interpreted as a design tool and not as a measure of level quality.

Finally, the one-factor-at-a-time analysis gives clear local sensitivity results but does not fully model interactions. The screening results already show that size, map type, enemies, items, and wall structure interact. A full factorial or model-based analysis could estimate these interactions more explicitly.

6.3 Possible Improvements

The most obvious and direct improvement would be to reduce the gap between planning and evaluation by using a more exact forward model or by exposing the environment itself as a cheaper simulation interface, making MCTS evaluations more trustworthy.

Another possible improvement could be to extend the generator from random parameterized generation toward search-based or optimization-based PCG [5]. Thus, instead of manually selecting profiles, the system could optimize for a multi-objective target like playability, skill-gap or structural diversity. This would naturally connect to evolutionary algorithms or other search methods.

A further improvement would be to add a human evaluation layer. Agent-based evaluation is scalable and reproducible, but cannot measure human-centric metrics like whether a map feels fun, fair or too challenging to human players. A controlled human study could compare perceived enjoyment or difficulty against agent-derived metrics.

Finally, the agent set itself could be expanded. We could include a stronger MCTS variant, an ablated MCTS without item activation, a learned policy or enemies with more strategic behaviors to name a few. These additions would make it clearer which properties of the generated maps are actually robust and which ones are agent-specific.

7 Conclusion

This project implemented a turn-based gridworld in which procedural map generation is evaluated through automated game-play agents. The system combines three fundamental game AI concepts: procedural content generation, search-based game AI through Monte Carlo Tree Search and empirical evaluation of agent performance.

The main finding is that the generator can produce content with meaningful differences in difficulty and pacing. Small maps are used for easy introductory

content, while medium and large maps show meaningful differences between the heuristic and MCTS-medium agents, as summarized in Table 6. The paired seed analysis shows how MCTS-medium solves many of the same generated maps that the heuristic fails to solve, proving that it is not only an aggregate effect.

The experiments show that win rate is not enough to describe generated content. Random-walk maps create more choke-points and longer games while arena maps tend to be more open. Combat parameters such as HP have very large effects, so the final validation fixed them and focused on generator-facing controls such as size, enemy profile, item profile, wall profile, and map type.

In conclusion, the project demonstrated a compact but complete PCG evaluation loop where we generate maps, validate them structurally, play them with agents, measure the outcomes and use the results to tune the generator. The current system is limited by its small game model, fixed agent set, separate MCTS forward model, and partly subjective tuning score, but it provides a solid foundation to study search-based procedural balancing, richer agents, and future human-centered evaluation.

8 Generative AI use

Generative AI tools were used as assistants during this project, mainly for code support, debugging, and language improvement. In particular, they were used to generate some of the visual assets for the Pygame-based user interface, including textures, icons, and other interface graphics and to improve the wording of parts of the report. All generated suggestions were checked, adapted, and validated by the authors before inclusion. The final implementation, experiments, and conclusions are the work of the authors.

References

1. Bamford, C.: Griddly: A platform for ai research in games. Software Impacts **8**, 100066 (2021). https://doi.org/10.1016/j.simpa.2021.100066
2. Browne, C.B., Powley, E., Whitehouse, D., Lucas, S.M., Cowling, P.I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., Colton, S.: A survey of monte carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in Games **4**(1), 1–43 (2012). https://doi.org/10.1109/TCIAIG.2012.2186810
3. Kocsis, L., Szepesvári, C.: Bandit based monte-carlo planning. In: Machine Learning: ECML 2006. pp. 282–293. Springer (2006). https://doi.org/10.1007/11871842_29
4. Shaker, N., Togelius, J., Nelson, M.J.: Procedural Content Generation in Games: A Textbook and an Overview of Current Research. Springer (2016). https://doi.org/10.1007/978-3-319-42716-4
5. Togelius, J., Yannakakis, G.N., Stanley, K.O., Browne, C.: Search-based procedural content generation: A taxonomy and survey. IEEE Transactions on Computational Intelligence and AI in Games **3**(3), 172–186 (2011). https://doi.org/10.1109/TCIAIG.2011.2148116

6. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association **22**(158), 209–212 (1927). https://doi.org/10.1080/01621459.1927.10502953
7. Yannakakis, G.N., Togelius, J.: Artificial Intelligence and Games. Springer (2018). https://doi.org/10.1007/978-3-319-63519-4